

UNIVERSITY OF DELHI

Department of Computer Science

COURSE NOTES

Guidelines for Information Security

NEP UGCF 2022 | Discipline Specific Core | Semester VIII

Based on: W. Stallings & L. Brown

Computer Security: Principles and Practice, 4th Edition

TABLE OF CONTENTS

UNIT 1: Overview of Computer Security	9
1.1 Computer Security Concepts.....	9
The CIA Triad	9
1.2 Threats, Attacks, and Assets	9
Security Threats	9
Security Attacks.....	9
Assets Requiring Protection.....	10
1.3 Security Functional Requirements	10
1.4 Fundamental Security Design Principles	10
UNIT 2: Cryptographic Tools.....	12
2.1 Confidentiality with Symmetric Encryption	12
What is Symmetric Encryption?	12
Types of Symmetric Ciphers	12
Advanced Encryption Standard (AES)	12
2.2 Message Authentication and Hash Functions.....	12
Message Authentication Code (MAC)	12
Cryptographic Hash Functions.....	12
HMAC (Hash-based MAC).....	13
2.3 Public-Key Encryption	13
Asymmetric Cryptography Principles	13
RSA Algorithm	13
Elliptic Curve Cryptography (ECC).....	13
2.4 Digital Signatures and Key Management.....	13
Digital Signatures.....	13
Key Management Issues	14
Diffie-Hellman Key Exchange	14
2.5 Random and Pseudorandom Numbers	14
True Random vs. Pseudorandom	14
Cryptographically Secure PRNG Requirements.....	14
2.6 Practical Application: Encryption of Stored Data.....	14
Full Disk Encryption (FDE).....	15
File and Database Encryption	15
Best Practices.....	15
UNIT 3: User Authentication and Access Control.....	16
3.1 Digital User Authentication Principles	16
Four Means of Authentication	16

Multi-Factor Authentication (MFA)	16
3.2 Password-Based Authentication	16
Password Vulnerabilities	16
Password Storage and Protection	16
Password Policy Recommendations	17
3.3 Remote User Authentication	17
Challenge-Response Protocols	17
Kerberos Authentication	17
3.4 Security Issues for User Authentication	17
3.5 Access Control Principles	18
Core Concepts	18
Access Control Models	18
3.6 Discretionary Access Control	18
Access Control Matrix	19
3.7 Example: UNIX File Access Control	19
Permission Model	19
Special Permissions	19
Limitations	19
3.8 Role-Based Access Control (RBAC)	19
RBAC Components	19
Role Hierarchy	19
Constraints	20
3.9 Attribute-Based Access Control (ABAC)	20
Advantages	20
Challenges	20
3.10 Identity, Credential, and Access Management (ICAM)	20
ICAM Framework Components	20
Single Sign-On (SSO)	20
3.11 Trust Frameworks	20
UNIT 4: Database and Data Center Security	22
4.1 The Need for Database Security	22
Why Database Security Matters	22
Unique Security Challenges	22
Database Security Threats	22
4.2 SQL Injection Attacks	23
What is SQL Injection?	23
Example Vulnerable Code	23
Types of SQL Injection	23

SQL Injection Impact	23
4.3 SQL Injection Countermeasures.....	23
1. Parameterized Queries (Prepared Statements)	23
2. Input Validation	24
3. Least Privilege for Database Accounts	24
4. Stored Procedures.....	24
5. Web Application Firewall (WAF)	24
6. Error Handling.....	24
7. Regular Security Testing.....	24
4.4 Database Access Control	24
Access Control Mechanisms.....	24
SQL Access Control Commands	25
Row-Level Security (RLS).....	25
Column-Level Security.....	25
Views for Access Control	25
4.5 Inference and Aggregation Problems	25
Inference Attack.....	25
Countermeasures	25
4.6 Database Encryption	25
Encryption Strategies.....	25
Key Management for Database Encryption.....	26
4.7 Database Auditing	26
What to Audit	26
Audit Trail Protection	26
UNIT 5: Software Security	27
5.1 Types of Malicious Software	27
5.2 Advanced Persistent Threat (APT)	27
Characteristics	27
APT Attack Lifecycle.....	28
5.3 Propagation — Infected Content (Viruses)	28
Virus Lifecycle	28
Virus Types.....	28
5.4 Propagation — Vulnerability Exploit (Worms)	28
Worm Characteristics	28
Famous Worms	28
Worm Propagation Methods	28
5.5 Propagation — Social Engineering (SPAM, Phishing)	29
SPAM Email	29

Phishing.....	29
Social Engineering Tactics.....	29
5.6 Payload — System Corruption	29
5.7 Payload — Attack Agent (Zombie, Bots).....	29
Zombie/Bot	29
Botnet	29
Botnet Uses	29
Botnet Communication	30
5.8 Payload — Information Theft.....	30
Keyloggers.....	30
Screen Scrapers	30
Spyware	30
Data Exfiltration Methods.....	30
5.9 Payload — Stealthing (Backdoors, Rootkits)	30
Backdoor	30
Rootkit	30
5.10 Malware Countermeasures.....	31
Prevention	31
Detection	31
Response and Recovery.....	31
5.11 Buffer Overflow Attacks	31
What is a Buffer Overflow?	31
Stack Overflow Attack.....	31
Heap Overflow	32
Integer Overflow	32
5.12 Defending Against Buffer Overflows	32
1. Safe Programming Practices	32
2. Stack Canaries	32
3. Address Space Layout Randomization (ASLR).....	32
4. Non-Executable Stack (NX bit / DEP)	32
5. Control Flow Integrity (CFI)	32
6. Compiler Protections.....	32
5.13 Handling Program Input.....	33
Input Validation Principles.....	33
Safe String Handling.....	33
Integer Overflow Prevention	33
5.14 Writing Safe Program Code.....	33
Secure Coding Principles.....	33

Code Review and Testing	33
Secure Development Lifecycle (SDL)	33
UNIT 6: Network Security	35
6.1 Denial-of-Service (DoS) Attacks	35
What is DoS?.....	35
DoS Attack Categories	35
6.2 Flooding Attacks.....	35
ICMP Flood (Ping Flood)	35
UDP Flood	35
TCP SYN Flood	35
HTTP Flood (Application Layer DoS)	36
6.3 Distributed Denial-of-Service (DDoS)	36
DDoS Architecture	36
Why DDoS is More Dangerous.....	36
DDoS Amplification Attacks	36
DDoS Mitigation.....	36
6.4 Intrusion Detection Systems (IDS).....	37
What is IDS?.....	37
IDS Types.....	37
Detection Methods.....	37
IDS vs IPS	37
IDS Challenges.....	38
6.5 Honeypots	38
What is a Honeypot?	38
Honeypot Uses	38
Honeypot Types	38
Honeynet	38
6.6 Firewalls.....	38
What is a Firewall?	38
Firewall Characteristics.....	38
Firewall Limitations	39
6.7 Firewall Access Policy	39
Policy Types	39
Rule Ordering	39
6.8 Types of Firewalls	39
1. Packet-Filtering Firewall.....	39
2. Stateful Inspection Firewall	39
3. Application-Level Gateway (Proxy Firewall)	40

4. Circuit-Level Gateway	40
5. Next-Generation Firewall (NGFW)	40
6.9 Firewall Deployment: DMZ Architecture	40
Demilitarized Zone (DMZ)	40
6.10 Public-Key Infrastructure (PKI)	41
What is PKI?	41
PKI Components.....	41
Certificate Trust Chain	41
PKI Use Cases	41
PKI Challenges.....	42
UNIT 7: Wireless, Cloud, and IoT Security.....	43
7.1 Cloud Computing.....	43
Cloud Service Models.....	43
Cloud Deployment Models.....	43
7.2 Cloud Security Concepts	43
Shared Responsibility Model	43
Cloud Security Threats	44
7.3 Cloud Security Approaches	44
Data Protection.....	44
Identity and Access Management (IAM)	44
Network Security.....	45
Logging and Monitoring	45
Compliance and Governance	45
Vendor Management	45
7.4 The Internet of Things (IoT)	45
What is IoT?	45
IoT Architecture Layers.....	45
7.5 IoT Security Challenges	46
Device-Level Challenges	46
Network-Level Challenges	46
Backend/Cloud Challenges.....	46
7.6 IoT Security Approaches	47
Secure Device Design	47
Secure Communication.....	47
Network Security.....	47
Cloud/Backend Security.....	47
Privacy Protection.....	47
Lifecycle Management.....	48

7.7 Wireless Security Overview	48
Wireless Security Threats	48
802.11 Security Evolution	48
WPA2/WPA3 Security Mechanisms.....	49
Wireless Security Best Practices	49
Enterprise Wireless Security (802.1X)	49
7.8 Mobile Device Security	49
Mobile Security Threats	49
Mobile Security Best Practices.....	50
iOS vs Android Security	50

UNIT 1: Overview of Computer Security

This unit introduces fundamental concepts of computer security, including the core principles, common threats and attacks, security requirements, and design principles that form the foundation of secure systems.

1.1 Computer Security Concepts

Computer security refers to the protection of computer systems and information from harm, theft, and unauthorized use. It involves protecting hardware, software, and data from cyber threats.

The CIA Triad

The three core security objectives are:

Confidentiality	Ensures that information is accessible only to those authorized to access it. Prevents unauthorized disclosure of sensitive data. Example: encryption of passwords.
Integrity	Ensures that information is accurate, complete, and has not been modified by unauthorized parties. Example: digital signatures, checksums.
Availability	Ensures that authorized users have timely and reliable access to information and resources when needed. Example: redundancy, backups, DoS protection.

Additional security concepts include:

- **Authenticity:** Verifies the identity of users and the origin of data
- **Accountability:** Ensures that actions can be traced to responsible parties (audit trails, logs)
- **Non-repudiation:** Prevents denial of actions performed (digital signatures provide proof)

1.2 Threats, Attacks, and Assets

Security Threats

A threat is a potential violation of security. Threats can be classified as:

- **Disclosure:** Unauthorized access to information (eavesdropping, packet sniffing)
- **Deception:** Acceptance of false data (spoofing, man-in-the-middle attacks)
- **Disruption:** Interruption or prevention of correct operation (denial of service)
- **Usurpation:** Unauthorized control of system resources (malware, privilege escalation)

Security Attacks

Attacks are actions that exploit vulnerabilities to realize threats:

Passive Attacks	Monitor or eavesdrop on communications without modification. Hard to detect. Examples: traffic analysis, release of message contents.
------------------------	---

Active Attacks	Modify data or create false data streams. Easier to detect but hard to prevent. Examples: masquerade, replay, message modification, denial of service.
-----------------------	--

Assets Requiring Protection

- Hardware: Physical devices (servers, routers, storage)
- Software: Operating systems, applications, utilities
- Data: Files, databases, configurations, authentication credentials
- Communication Lines: Network connections, wireless links
- Human Resources: Personnel with access to systems

1.3 Security Functional Requirements

Security functional requirements define what a system must do to provide security. The FIPS 200 standard identifies 17 security-related areas:

1. Access Control: Limit system access to authorized users
2. Awareness and Training: Ensure users understand security policies
3. Audit and Accountability: Create and maintain system audit records
4. Certification and Accreditation: Periodic security assessment
5. Configuration Management: Establish and maintain baseline configurations
6. Contingency Planning: Emergency response, backup, disaster recovery
7. Identification and Authentication: Uniquely identify and authenticate users
8. Incident Response: Prepare for and respond to security incidents
9. Maintenance: Perform periodic system maintenance securely
10. Media Protection: Protect physical media containing sensitive information
11. Physical and Environmental Protection: Limit physical access to systems
12. Planning: Develop and implement security plans
13. Personnel Security: Screen and train personnel
14. Risk Assessment: Periodically assess security risks
15. System and Services Acquisition: Include security in acquisition process
16. System and Communications Protection: Monitor and control communications
17. System and Information Integrity: Identify and correct flaws promptly

1.4 Fundamental Security Design Principles

These principles guide the design of secure systems:

Economy of Mechanism	Keep designs simple and small. Simpler systems are easier to verify and less likely to contain vulnerabilities.
Fail-Safe Defaults	Base access decisions on permission rather than exclusion. Default to lack of access; explicitly grant access.

Complete Mediation	Check every access to every object. Don't cache or bypass authorization checks.
Open Design	Security should not depend on secrecy of design or implementation. Use strong algorithms, not obscurity.
Separation of Privilege	Require multiple conditions for access. Example: two-factor authentication.
Least Privilege	Give programs and users only the minimum privileges needed to complete their tasks.
Least Common Mechanism	Minimize shared resources and mechanisms. Reduces information flow between users.
Psychological Acceptability	Security mechanisms should be easy to use; otherwise users will circumvent them.
Isolation	Separate users from each other, and processes from each other. Use virtualization, sandboxing.
Encapsulation	Hide internal implementation details. Provide clean interfaces.
Modularity	Use modular architecture where components can be independently updated.
Layering	Use multiple overlapping protection mechanisms (defense in depth).

Key Takeaway: Security is not a single technology but a combination of policies, procedures, and technical mechanisms working together to protect assets from threats and attacks.

UNIT 2: Cryptographic Tools

Cryptography provides the mathematical foundation for securing information. This unit covers encryption techniques, authentication mechanisms, and key management strategies.

2.1 Confidentiality with Symmetric Encryption

What is Symmetric Encryption?

Symmetric encryption uses the same key for both encryption and decryption. It is fast and efficient for large volumes of data.

Plaintext	Original readable message (P)
Ciphertext	Encrypted message (C)
Encryption	$C = E(K, P)$ — transform plaintext using key K
Decryption	$P = D(K, C)$ — recover plaintext using same key K

Types of Symmetric Ciphers

Block Ciphers: Encrypt fixed-size blocks (e.g., AES with 128-bit blocks). Modes include ECB, CBC, CTR, GCM.

Stream Ciphers: Encrypt one bit or byte at a time (e.g., RC4, ChaCha20). Suitable for streaming data.

Advanced Encryption Standard (AES)

- Block size: 128 bits
- Key sizes: 128, 192, or 256 bits
- Based on Substitution-Permutation Network (SPN)
- Operations: SubBytes, ShiftRows, MixColumns, AddRoundKey
- Industry standard for confidentiality

2.2 Message Authentication and Hash Functions

Message Authentication Code (MAC)

MAC provides both integrity and authenticity. It is a cryptographic checksum generated using a secret key.

$MAC = C(K, M)$ where M is the message and K is the shared key.

The receiver computes MAC using the same key and compares it with the received MAC.

Cryptographic Hash Functions

A hash function H produces a fixed-size output (hash or digest) from variable-length input.

Properties of a secure hash function:

- Preimage resistance: Given h , it is computationally infeasible to find M such that $H(M) = h$
- Second preimage resistance: Given M_1 , infeasible to find $M_2 \neq M_1$ such that $H(M_1) = H(M_2)$
- Collision resistance: Infeasible to find any two messages M_1, M_2 such that $H(M_1) = H(M_2)$

Common hash functions:

- SHA-256: 256-bit output, part of SHA-2 family
- SHA-3: Latest standard, based on Keccak
- MD5, SHA-1: Deprecated due to collision vulnerabilities

HMAC (Hash-based MAC)

$HMAC = H((K \oplus opad) \parallel H((K \oplus ipad) \parallel M))$

Combines hash function with a secret key for message authentication.

2.3 Public-Key Encryption

Asymmetric Cryptography Principles

Uses two mathematically related keys:

- Public Key (PU): Known to everyone, used for encryption
- Private Key (PR): Kept secret, used for decryption

Encryption: $C = E(PU, P)$

Decryption: $P = D(PR, C)$

RSA Algorithm

Based on the difficulty of factoring large numbers.

Key Generation:

18. Select two large prime numbers p and q
19. Compute $n = p \times q$
20. Compute $\phi(n) = (p - 1)(q - 1)$
21. Choose e such that $1 < e < \phi(n)$ and $\gcd(e, \phi(n)) = 1$
22. Compute d such that $d \times e \equiv 1 \pmod{\phi(n)}$
23. Public key: (e, n) , Private key: (d, n)

Encryption: $C = P^e \bmod n$

Decryption: $P = C^d \bmod n$

Elliptic Curve Cryptography (ECC)

- Provides equivalent security with smaller key sizes than RSA
- 256-bit ECC key $\approx$ 3072-bit RSA key
- Efficient for mobile and IoT devices
- Used in ECDH (key exchange) and ECDSA (digital signatures)

2.4 Digital Signatures and Key Management

Digital Signatures

Provide authentication, integrity, and non-repudiation.

Signing: Sign with sender's private key

$S = \text{Encrypt}(\text{PR_sender}, H(M))$

Verification: Decrypt with sender's public key

$H(M) = \text{Decrypt}(\text{PU_sender}, S)$

If hashes match, signature is valid.

Key Management Issues

- Key Generation: Use cryptographically secure random number generators
- Key Distribution: How to securely share keys (Diffie-Hellman, PKI)
- Key Storage: Protect keys using hardware security modules (HSMs), key encryption keys
- Key Rotation: Periodically change keys to limit exposure
- Key Revocation: Handle compromised keys (Certificate Revocation Lists, OCSP)

Diffie-Hellman Key Exchange

Allows two parties to establish a shared secret over an insecure channel.

24. Alice and Bob agree on public parameters: prime p and generator g
25. Alice chooses secret a , computes $A = g^a \bmod p$, sends A to Bob
26. Bob chooses secret b , computes $B = g^b \bmod p$, sends B to Alice
27. Alice computes $K = B^a \bmod p$
28. Bob computes $K = A^b \bmod p$
29. Both arrive at the same shared secret $K = g^{(ab)} \bmod p$

2.5 Random and Pseudorandom Numbers

Randomness is critical for:

- Key generation
- Initialization vectors (IVs) for encryption
- Nonces to prevent replay attacks
- Salt for password hashing

True Random vs. Pseudorandom

TRNG (True RNG)	Uses physical processes (thermal noise, radioactive decay). Unpredictable but slow.
PRNG (Pseudorandom)	Uses deterministic algorithms seeded with random input. Fast but must be cryptographically secure.

Cryptographically Secure PRNG Requirements

- Unpredictability: Cannot predict next output from previous outputs
- Pass statistical tests: Output should appear random
- Proper seeding: Use sufficient entropy from TRNG

2.6 Practical Application: Encryption of Stored Data

Full Disk Encryption (FDE)

- Encrypts entire disk or partition (BitLocker, FileVault, LUKS)
- Transparent to user after authentication
- Protects against physical theft
- Limitations: Does not protect against online attacks when system is running

File and Database Encryption

- Selective encryption of sensitive files or database columns
- Application-level encryption: Data encrypted before storage
- Key management is critical: store keys separately from data

Best Practices

- Use strong algorithms: AES-256, RSA-2048 or higher
- Never implement custom cryptography
- Use authenticated encryption modes (AES-GCM)
- Securely manage keys: use HSMs, key derivation functions (KDFs)
- Combine symmetric and asymmetric: use public-key for key exchange, symmetric for bulk data

Key Takeaway: Cryptographic tools provide confidentiality through encryption, integrity and authenticity through MACs and hash functions, and non-repudiation through digital signatures. Proper key management is essential for all cryptographic systems.

UNIT 3: User Authentication and Access Control

This unit covers mechanisms for verifying user identity and controlling access to system resources, forming the backbone of security policy enforcement.

3.1 Digital User Authentication Principles

Four Means of Authentication

Authentication verifies the identity of a user based on:

Something you know	Passwords, PINs, security questions
Something you have	Smart cards, tokens, mobile devices
Something you are	Biometrics: fingerprints, iris scans, facial recognition
Something you do	Behavioral biometrics: typing patterns, gait, voice

Multi-Factor Authentication (MFA)

Combines two or more authentication factors for stronger security.

- Two-Factor Authentication (2FA): Password + OTP from mobile device
- Reduces risk of credential compromise
- Required for high-security environments

3.2 Password-Based Authentication

Password Vulnerabilities

- Weak passwords: Short, predictable, dictionary words
- Password reuse: Same password across multiple sites
- Phishing: Tricking users into revealing passwords
- Brute force: Trying all possible combinations
- Dictionary attacks: Trying common words and variations
- Rainbow tables: Precomputed hashes of common passwords

Password Storage and Protection

Never store passwords in plaintext!

Hash and Salt:

30. Generate random salt for each user
31. Compute hash = $H(\text{password} \parallel \text{salt})$
32. Store (salt, hash) in database
33. At login: retrieve salt, compute hash of entered password, compare

Use slow hash functions (bcrypt, scrypt, Argon2) to resist brute force.

Password Policy Recommendations

- Minimum length: 12-16 characters
- Complexity: Mix of uppercase, lowercase, digits, symbols
- Avoid common patterns: No keyboard walks (qwerty), sequences (12345)
- Periodic change: Every 60-90 days for sensitive systems
- Account lockout: After N failed attempts
- Password history: Prevent reuse of recent passwords

3.3 Remote User Authentication

Challenge-Response Protocols

Prevent replay attacks by using fresh challenges.

Protocol:

34. Server sends random challenge C
35. Client computes response $R = f(C, \text{shared_secret})$
36. Server verifies R using the same function

Kerberos Authentication

Centralized authentication service for distributed systems. Uses symmetric cryptography and trusted third party (KDC - Key Distribution Center).

Components:

- Authentication Server (AS): Issues Ticket-Granting Tickets (TGTs)
- Ticket-Granting Server (TGS): Issues service tickets
- Client: User requesting service
- Service Server: Resource being accessed

Protocol flow:

37. User logs in, AS authenticates and issues TGT (encrypted with user's password)
38. Client decrypts TGT, sends it to TGS when requesting a service
39. TGS issues service ticket (ST)
40. Client presents ST to service server
41. Service validates ticket and grants access

Advantages: Single sign-on, mutual authentication, time-limited tickets

Limitations: Requires synchronized clocks, single point of failure (KDC)

3.4 Security Issues for User Authentication

- Credential Theft: Keyloggers, shoulder surfing, phishing
- Man-in-the-Middle: Attacker intercepts authentication

- Replay Attacks: Captured credentials replayed later (use timestamps, nonces)
- Denial of Service: Account lockout attacks
- Social Engineering: Manipulating users to reveal credentials
- Session Hijacking: Stealing session tokens after authentication

Countermeasures: MFA, encrypted channels (TLS), challenge-response, session management, user education.

3.5 Access Control Principles

Access control enforces security policy by regulating who can access what resources.

Core Concepts

Subject	Active entity requesting access (user, process, program)
Object	Passive resource being accessed (file, database, printer)
Access Right	Permission to perform operation (read, write, execute, delete)

Access Control Models

1. Discretionary Access Control (DAC):
 - Owner of resource controls access
 - Flexible but can lead to privilege creep
 - Example: UNIX file permissions (owner, group, others)
2. Mandatory Access Control (MAC):
 - System enforces access based on security labels
 - Users cannot change permissions
 - Used in military and government (Top Secret, Secret, Confidential)
 - Example: SELinux, Bell-LaPadula model
3. Role-Based Access Control (RBAC):
 - Access based on user's role in organization
 - Roles assigned permissions, users assigned roles
 - Easier to manage in large organizations
 - Example: Admin role, Manager role, Employee role
4. Attribute-Based Access Control (ABAC):
 - Access based on attributes of subject, object, and environment
 - Fine-grained, flexible policies
 - Example: Allow access if (user.department == 'HR' AND file.classification == 'internal' AND time.hour < 18)

3.6 Discretionary Access Control

Access Control Matrix

Conceptual model: rows = subjects, columns = objects, cells = access rights.

Implementation approaches:

- Access Control Lists (ACL): Column-wise, stored with object. Lists subjects and their permissions for that object.
- Capability Lists: Row-wise, stored with subject. Lists objects the subject can access and permissions.

3.7 Example: UNIX File Access Control

Permission Model

Each file has three permission sets:

- Owner (u): File creator
- Group (g): Defined group of users
- Others (o): Everyone else

Each set has three permissions:

- Read (r): View file contents or list directory
- Write (w): Modify file or add/remove directory entries
- Execute (x): Run file as program or enter directory

Represented as: rwxr-xr-- (owner: rwx, group: r-x, others: r--)

Numeric notation: 754 (owner: 7=rwx, group: 5=r-x, others: 4=r--)

Special Permissions

- SetUID (s): Execute with owner's privileges
- SetGID (s): Execute with group's privileges
- Sticky bit (t): Only owner can delete files in directory

Limitations

- Coarse granularity: Only three permission sets
- Cannot specify access for specific users outside owner/group
- No negative permissions (deny rules)
- Solution: Extended ACLs allow finer control

3.8 Role-Based Access Control (RBAC)

RBAC Components

- Users: Individual accounts
- Roles: Job functions (e.g., Doctor, Nurse, Admin)
- Permissions: Operations on objects (e.g., read_patient_record)
- Sessions: User activates subset of assigned roles

Role Hierarchy

Senior roles inherit permissions from junior roles.

Example: Senior Doctor inherits all Doctor permissions + additional privileges.

Constraints

- Mutually exclusive roles: User cannot be both Auditor and Administrator (separation of duties)
- Cardinality: Maximum number of users per role
- Prerequisites: Must have Role A to get Role B

3.9 Attribute-Based Access Control (ABAC)

ABAC policies use Boolean logic on attributes:

Policy example: ALLOW if (subject.clearance >= object.classification AND subject.department == object.owner_department AND environment.location == 'secure_facility')

Advantages

- Fine-grained, context-aware access control
- Dynamic policies based on real-time attributes
- Reduces number of explicit rules

Challenges

- Complex policy management
- Performance overhead for attribute evaluation
- Requires consistent attribute management

3.10 Identity, Credential, and Access Management (ICAM)

ICAM Framework Components

- Identity Management: Provisioning, deprovisioning, lifecycle management
- Credential Management: Issuing, renewing, revoking credentials (passwords, tokens, certificates)
- Access Management: Authentication, authorization, auditing
- Federation: Trusting identities across organizational boundaries (SAML, OAuth, OpenID Connect)

Single Sign-On (SSO)

User authenticates once and gains access to multiple systems.

- Improves user experience
- Centralized authentication
- Risk: Single point of compromise

3.11 Trust Frameworks

Define rules and standards for identity verification and trust relationships between organizations.

- X.509 PKI: Certificate-based trust model
- SAML (Security Assertion Markup Language): XML-based identity federation
- OAuth 2.0: Authorization framework for delegated access
- OpenID Connect: Authentication layer on top of OAuth 2.0

Key Takeaway: Strong authentication verifies who users are, while access control determines what they can do. Together they enforce security policies through identity management, proper credential handling, and policy-based authorization.

UNIT 4: Database and Data Center Security

Databases store critical organizational data and present unique security challenges due to their complexity, granular access requirements, and vulnerability to sophisticated attacks.

4.1 The Need for Database Security

Why Database Security Matters

- Databases contain sensitive information: customer data, financial records, intellectual property
- Single breach can expose millions of records
- Regulatory requirements: GDPR, HIPAA, PCI-DSS mandate data protection
- High-value target for attackers: monetizable data (credit cards, personal info)

Unique Security Challenges

Databases present challenges beyond OS security:

- Granular access control: Need to control access at table, row, and column level
- Multi-user concurrency: Many users accessing simultaneously
- Complex query language: SQL injection vulnerabilities
- Inference and aggregation: Users may infer sensitive info from authorized queries
- Data at multiple sensitivity levels: Mixed classification in same database
- Backup and recovery: Backup files contain sensitive data

Database Security Threats

Excessive Privileges	Users granted more access than needed (violates least privilege)
Privilege Abuse	Legitimate users misusing authorized access for malicious purposes
SQL Injection	Attacker injects malicious SQL code through application inputs
Malware	Database-specific malware to steal data or manipulate records
Weak Audit Trail	Insufficient logging makes it hard to detect or investigate breaches
Storage Media Exposure	Unencrypted backup tapes, decommissioned hard drives
Unmanaged Sensitive Data	Unknown locations of sensitive data in database (data sprawl)

4.2 SQL Injection Attacks

What is SQL Injection?

SQL injection (SQLi) is a code injection attack where attacker inserts malicious SQL statements into application input fields to manipulate database queries.

Example Vulnerable Code

Consider a login form with username and password:

```
SELECT * FROM users WHERE username = '<user_input>' AND password = '<password_input>'
```

Normal input: username = 'alice', password = 'secret123'

Query becomes: `SELECT * FROM users WHERE username = 'alice' AND password = 'secret123'`

Malicious input: username = ' OR '1'='1' --', password = 'anything'

Query becomes: `SELECT * FROM users WHERE username = " OR '1'='1' --' AND password = 'anything'`

The -- comments out the password check, and '1'='1' is always true, so query returns all users.

Types of SQL Injection

1. In-Band SQLi (Classic):

- Attacker uses same channel for attack and results
- Error-based: Trigger database errors to extract information
- UNION-based: Use UNION operator to combine results from multiple queries

Example: ' UNION SELECT credit_card_number FROM payment_info --

2. Inferential SQLi (Blind):

- No direct data transfer; attacker infers information from application behavior
- Boolean-based: Ask true/false questions based on response changes
- Time-based: Use database delays (SLEEP, WAITFOR) to infer results

Example: ' AND IF(SUBSTRING(password,1,1)='a', SLEEP(5), 0) --

3. Out-of-Band SQLi:

- Uses different channel for exfiltration (DNS, HTTP requests)
- Rare; requires specific database features enabled

SQL Injection Impact

- Data theft: Extract entire database contents
- Authentication bypass: Login without credentials
- Data manipulation: Modify or delete records
- Privilege escalation: Gain admin access
- Denial of service: Drop tables, crash database
- Remote code execution: In some cases, execute OS commands

4.3 SQL Injection Countermeasures

1. Parameterized Queries (Prepared Statements)

Use placeholders for user input; database treats input as data, not code.

Vulnerable: `SELECT * FROM users WHERE id = '<userId>'`

Secure: PreparedStatement: `SELECT * FROM users WHERE id = ?`
Then bind `userId` to the placeholder. Database will not interpret it as SQL.

2. Input Validation

- Whitelist allowed characters (e.g., only alphanumeric for usernames)
- Reject or sanitize special SQL characters: `'`, `"`, `--`, `;`, `/*`, `*/`
- Validate data type: ensure numeric fields contain only numbers
- Limit input length

3. Least Privilege for Database Accounts

- Application should use database account with minimal permissions
- Read-only for queries, no DROP, ALTER permissions
- Separate accounts for different application components

4. Stored Procedures

Encapsulate SQL logic in database; reduce dynamic query construction.

BUT: Still vulnerable if stored procedure itself builds queries dynamically. Must use parameterization inside stored procedures too.

5. Web Application Firewall (WAF)

Filter malicious HTTP requests before they reach application. Can detect common SQLi patterns.

6. Error Handling

- Do not expose detailed database errors to users
- Generic error messages prevent information disclosure
- Log errors securely for administrators

7. Regular Security Testing

- Penetration testing
- Automated SQLi scanning tools
- Code review for vulnerable patterns

4.4 Database Access Control

Access Control Mechanisms

1. Discretionary Access Control (DAC):

- SQL GRANT and REVOKE statements
- Table owner grants privileges to other users
- Example: `GRANT SELECT, INSERT ON employees TO hr_user`

2. Mandatory Access Control (MAC):

- Assign security labels to data and users
- System enforces label-based access rules
- Example: User with 'Secret' clearance cannot access 'Top Secret' data

3. Role-Based Access Control (RBAC):

- Create roles (e.g., `HR_Manager`, `Sales_Rep`)

- Grant permissions to roles, assign roles to users
- Simplifies management in large databases

SQL Access Control Commands

```
CREATE ROLE sales_team;
GRANT SELECT ON products TO sales_team;
GRANT INSERT, UPDATE ON orders TO sales_team;
GRANT sales_team TO alice, bob;
```

Row-Level Security (RLS)

Control access at individual row level based on user attributes.

Example: Sales reps can only view orders from their region.

Implementation: Use WHERE clause filters based on session user.

Column-Level Security

Control access to specific columns.

Example: Only HR can view salary column in employees table.

```
GRANT SELECT(name, department) ON employees TO general_users;
```

Views for Access Control

Create views that filter sensitive data, grant access to views instead of base tables.

Example: `CREATE VIEW public_employees AS SELECT name, department FROM employees;` (hides salary)

4.5 Inference and Aggregation Problems

Inference Attack

User infers sensitive information from authorized queries without directly accessing restricted data.

Example: A user knows average salary of department is \$80,000 and there are 5 employees. After one employee leaves, average becomes \$85,000. User can calculate departed employee's salary.

Countermeasures

- Query restriction: Limit aggregation queries that involve small groups
- Perturbation: Add noise to query results
- Cell suppression: Hide results for small sample sizes
- Audit query sequences: Detect suspicious patterns

4.6 Database Encryption

Encryption Strategies

1. Transparent Data Encryption (TDE):

- Encrypts entire database at file level
- Transparent to applications
- Protects against physical theft of storage media
- Does not protect against SQL injection or insider threats

2. Column-Level Encryption:

- Encrypt specific sensitive columns (e.g., credit card numbers)
- Application-level encryption before storing in database
- Finer control but requires key management

3. Application-Level Encryption:

- Application encrypts data before sending to database
- Maximum control and security
- Cannot perform searches or indexing on encrypted data (unless using searchable encryption schemes)

Key Management for Database Encryption

- Store encryption keys separately from database
- Use Hardware Security Modules (HSMs) for key storage
- Rotate keys periodically
- Control access to keys using strict policies

4.7 Database Auditing

Audit trails record database activities for accountability and forensics.

What to Audit

- Authentication attempts (success and failure)
- All privileged operations (GRANT, DROP, ALTER)
- Access to sensitive tables/columns
- Data modifications (INSERT, UPDATE, DELETE) with before/after values
- Schema changes
- Query execution (especially failed queries, may indicate SQLi attempts)

Audit Trail Protection

- Store audit logs separately from main database
- Write-only access for applications (prevent tampering)
- Regular review and analysis
- Retention policy compliant with regulations

Key Takeaway: Database security requires defense in depth: input validation and parameterized queries prevent SQL injection, access control enforces least privilege, encryption protects data at rest, and auditing enables detection and forensics.

UNIT 5: Software Security

Software vulnerabilities enable most cyber attacks. This unit examines malicious software types, propagation mechanisms, and secure coding practices.

5.1 Types of Malicious Software

Malware (malicious software) is software designed to infiltrate, damage, or perform unauthorized actions on a computer system.

Virus	Malware that replicates by infecting other programs. Requires user action to execute.
Worm	Self-replicating malware that spreads automatically over networks without user intervention.
Trojan	Appears legitimate but contains hidden malicious functionality. Does not replicate.
Spyware	Secretly monitors user activity and collects information (keystrokes, browsing, credentials).
Rootkit	Conceals malware presence by modifying OS functions. Hides processes, files, network connections.
Backdoor	Provides unauthorized remote access, bypassing normal authentication.
Ransomware	Encrypts files and demands payment for decryption key.
Adware	Displays unwanted advertisements, may track user behavior.

5.2 Advanced Persistent Threat (APT)

APT is a prolonged, targeted cyberattack where attackers gain unauthorized access and remain undetected for an extended period.

Characteristics

- **Advanced:** Uses sophisticated techniques, custom malware, zero-day exploits
- **Persistent:** Maintains long-term presence, establishes multiple access points
- **Threat:** Well-funded, organized groups (nation-states, cyber criminals)
- **Targeted:** Specific organization or industry

- **Stealthy:** Avoids detection, mimics normal traffic

APT Attack Lifecycle

42. Initial intrusion: Spear phishing, watering hole attacks, exploiting vulnerabilities
43. Establish foothold: Install backdoor, create persistence mechanisms
44. Escalate privileges: Exploit local vulnerabilities to gain admin access
45. Internal reconnaissance: Map network, identify valuable assets
46. Lateral movement: Spread to other systems, compromise additional accounts
47. Maintain presence: Install additional backdoors, rootkits
48. Complete mission: Exfiltrate data, destroy evidence

5.3 Propagation — Infected Content (Viruses)

Virus Lifecycle

49. Dormant: Virus inactive, waiting for trigger condition
50. Propagation: Virus replicates itself, infects other files/programs
51. Triggering: Activated by event (date, action, user command)
52. Execution: Delivers payload (damage, data theft, etc.)

Virus Types

1. Parasitic Virus: Attaches to executable files. Runs when host program executes.
2. Memory-Resident Virus: Lodges in RAM, infects programs as they run.
3. Boot Sector Virus: Infects master boot record. Executes during system boot.
4. Macro Virus: Infects documents with macros (Word, Excel). Spreads through document sharing.
5. Polymorphic Virus: Changes code each time it replicates (mutates signature to evade detection).
6. Metamorphic Virus: Completely rewrites itself with each infection. Harder to detect than polymorphic.

5.4 Propagation — Vulnerability Exploit (Worms)

Worm Characteristics

- **Self-propagating:** Spreads automatically without user action
- **Exploits vulnerabilities:** Targets unpatched systems
- **Network-based:** Scans for vulnerable hosts, infects remotely
- **Fast spread:** Can infect thousands of systems in hours

Famous Worms

- Morris Worm (1988): Exploited UNIX vulnerabilities, caused widespread DoS
- Code Red (2001): Exploited IIS buffer overflow, defaced websites
- Conficker (2008): Exploited Windows vulnerability, created massive botnet
- WannaCry (2017): Ransomware worm using EternalBlue exploit

Worm Propagation Methods

- **Network scanning:** Randomly or sequentially scan IP ranges for vulnerable hosts
- **Email propagation:** Send copies via email to contacts (worm-like viruses)

- File sharing: Spread through shared folders, removable media
- Exploiting services: Target vulnerable network services (SMB, RDP, SSH)

5.5 Propagation — Social Engineering (SPAM, Phishing)

SPAM Email

Unsolicited bulk email. Used to spread malware, phishing links, scams.

Phishing

Fraudulent attempt to obtain sensitive information by masquerading as trustworthy entity.

Types:

- Email phishing: Fake emails from banks, services asking for credentials
- Spear phishing: Targeted phishing using personalized information
- Whaling: Phishing targeting high-level executives
- Smishing: Phishing via SMS
- Vishing: Voice phishing via phone calls

Example: Email claiming 'Your account will be closed unless you verify' with link to fake login page.

Social Engineering Tactics

- Urgency: 'Act now or account will be locked'
- Authority: Impersonate CEO, IT admin, law enforcement
- Fear: Threaten negative consequences
- Curiosity: 'You won a prize, click here'
- Trust: Impersonate known contact or brand

5.6 Payload — System Corruption

Malware payloads that damage or disrupt systems:

- Data destruction: Delete files, format drives, overwrite data
- Data encryption: Ransomware encrypts files, demands payment
- Logic bomb: Malicious code triggered by specific condition (date, event)
- System modification: Change configurations, disable security software
- Denial of Service: Consume resources, crash system

5.7 Payload — Attack Agent (Zombie, Bots)

Zombie/Bot

Compromised computer under remote control of attacker.

Botnet

Network of infected machines controlled by botmaster (Command & Control server).

Botnet Uses

- Distributed Denial of Service (DDoS): Coordinate attacks from thousands of bots
- SPAM distribution: Send bulk email

- Click fraud: Generate fake ad clicks
- Cryptocurrency mining: Use victim's CPU/GPU
- Credential stuffing: Test stolen credentials across websites
- Hosting illegal content: Use bots as servers

Botnet Communication

- Centralized: Bots connect to C&C server (single point of failure)
- P2P: Bots communicate with each other (more resilient)
- Domain fluxing: C&C domain changes frequently to evade blocking

5.8 Payload — Information Theft

Keyloggers

Record keystrokes to capture passwords, credit cards, messages.

Types: Software (runs in background) or Hardware (physical device on keyboard cable).

Screen Scrapers

Capture screenshots periodically or on specific events.

Spyware

Broad category: monitors user activity, collects data without consent.

- Tracks browsing history, searches
- Records application usage
- Accesses camera, microphone
- Harvests contacts, emails

Data Exfiltration Methods

- Upload to attacker's server
- Email to attacker
- DNS tunneling: Encode data in DNS queries
- Steganography: Hide data in images

5.9 Payload — Stealthing (Backdoors, Rootkits)

Backdoor

Mechanism for bypassing normal authentication to gain remote access.

- Installed by malware or left by developers (debug code)
- Listens on network port for attacker commands
- Example: NetBus, Back Orifice

Rootkit

Malware that hides its presence and other malware by modifying OS functions.

Rootkit Levels:

- User-mode rootkit: Modifies application-level processes
- Kernel-mode rootkit: Modifies kernel (deeper access, harder to detect)

- Bootkit: Infects bootloader (executes before OS)
- Firmware rootkit: Infects BIOS/UEFI (persists even after OS reinstall)

Rootkit Techniques:

- Hooking: Intercept system calls, hide files/processes/network connections
- Direct Kernel Object Manipulation (DKOM): Modify kernel data structures
- Filter driver: Insert driver to hide activities

5.10 Malware Countermeasures

Prevention

- Antivirus software: Signature-based detection, heuristic analysis
- Keep software updated: Patch vulnerabilities promptly
- Least privilege: Run with minimal permissions
- Firewall: Block unauthorized network connections
- Email filtering: Block attachments, suspicious links
- Disable macros: In documents from untrusted sources
- User education: Recognize phishing, social engineering

Detection

- Intrusion Detection Systems (IDS): Monitor network/system for suspicious activity
- Behavioral analysis: Detect anomalies in system behavior
- Integrity checking: Verify files haven't been modified (checksums)
- Monitoring: Track network traffic, process activity, file changes
- Sandbox: Execute suspicious files in isolated environment

Response and Recovery

- Isolate infected systems: Prevent spread
- Analyze malware: Reverse engineer to understand behavior
- Remove malware: Use specialized removal tools
- Restore from clean backups: After confirming malware elimination
- Update defenses: Adjust rules, signatures based on incident

5.11 Buffer Overflow Attacks

What is a Buffer Overflow?

Occurs when program writes more data to a buffer than it can hold, overwriting adjacent memory.

Stack Overflow Attack

Exploit buffer overflow in stack-allocated variables to overwrite return address and execute malicious code.

Stack structure during function call:

- Local variables
- Saved frame pointer
- Return address (where to jump after function returns)

- Function arguments

Attack steps:

53. Find vulnerable function with buffer on stack (e.g., strcpy, gets)
54. Craft input longer than buffer size
55. Overflow buffer, overwrite return address with address of shellcode
56. When function returns, jumps to shellcode instead of legitimate code
57. Shellcode executes (often spawns shell with attacker's control)

Example vulnerable code:

```
void vulnerable(char *input) {
    char buffer[64];
    strcpy(buffer, input); // No bounds checking!
}
```

Heap Overflow

Overwrite heap metadata or function pointers stored on heap. Can lead to arbitrary code execution.

Integer Overflow

Arithmetic operation produces value outside representable range.

Example: unsigned char x = 255; x = x + 1; // x becomes 0

Can lead to buffer overflows if used in size calculations.

5.12 Defending Against Buffer Overflows

1. Safe Programming Practices

- Use safe string functions: strncpy, strlcpy instead of strcpy
- Bounds checking: Validate input sizes
- Avoid dangerous functions: gets, scanf, sprintf
- Use safer languages: Java, Python (automatic bounds checking)

2. Stack Canaries

Place random value (canary) on stack between buffer and return address. Check canary before function returns. If modified, overflow detected, terminate program.

3. Address Space Layout Randomization (ASLR)

Randomize memory locations of stack, heap, libraries. Attacker cannot predict addresses, making exploit harder.

4. Non-Executable Stack (NX bit / DEP)

Mark stack memory as non-executable. Prevents execution of shellcode placed on stack.

5. Control Flow Integrity (CFI)

Ensure program follows legitimate control flow. Prevent jumps to arbitrary addresses.

6. Compiler Protections

- Stack protection: -fstack-protector in GCC

- Fortify source: `-D_FORTIFY_SOURCE` (adds runtime checks)
- Position Independent Executables (PIE): Supports ASLR

5.13 Handling Program Input

Input Validation Principles

- Whitelist validation: Accept only known good input (preferred)
- Blacklist validation: Reject known bad input (incomplete, can be bypassed)
- Canonicalization: Convert input to standard form before validation
- Length checks: Enforce maximum input size
- Type checking: Ensure input matches expected data type
- Range checking: Validate numeric inputs are within acceptable range

Safe String Handling

Use functions that accept buffer size:

- `strncpy(dest, src, n)`: Copy at most `n` characters
- `snprintf(dest, n, format, ...)`: Formatted output with size limit
- `fgets(buffer, size, stdin)`: Read with size limit

NEVER use: `gets`, `strcpy`, `sprintf`, `scanf` without width limits.

Integer Overflow Prevention

- Check before arithmetic: `if (a + b < a) // overflow occurred`
- Use larger data types for intermediate calculations
- Compiler warnings: Enable `-Wconversion`, `-Woverflow`
- Safe integer libraries: `SafeInt` (C++)

5.14 Writing Safe Program Code

Secure Coding Principles

- Principle of least privilege: Run with minimum necessary permissions
- Fail securely: Default to denial, safe error handling
- Defense in depth: Multiple layers of security
- Keep it simple: Complex code has more vulnerabilities
- Don't trust input: Validate all external data
- Avoid security by obscurity: Use proven algorithms

Code Review and Testing

- Static analysis: Automated tools to find vulnerabilities (e.g., Coverity, SonarQube)
- Dynamic analysis: Runtime testing, fuzzing
- Peer review: Manual code inspection by other developers
- Penetration testing: Simulate attacks to find weaknesses

Secure Development Lifecycle (SDL)

58. Training: Educate developers on secure coding
59. Requirements: Define security requirements

- 60. Design: Threat modeling, security architecture
- 61. Implementation: Follow secure coding guidelines
- 62. Verification: Testing, code review, security scans
- 63. Release: Final security review, incident response plan
- 64. Response: Monitor, patch vulnerabilities post-release

Key Takeaway: Software security requires understanding attack vectors (malware types, buffer overflows, social engineering) and implementing defenses through secure coding practices, input validation, memory protection mechanisms, and secure development processes.

UNIT 6: Network Security

Network security protects data in transit and defends against network-based attacks. This unit covers DoS attacks, intrusion detection, firewalls, and public-key infrastructure.

6.1 Denial-of-Service (DoS) Attacks

What is DoS?

Denial-of-Service attack makes a service unavailable to legitimate users by overwhelming resources or exploiting vulnerabilities.

DoS Attack Categories

1. Bandwidth Consumption:

- Flood victim with massive traffic to saturate network link
- Victim cannot receive legitimate traffic
- Example: UDP flood, ICMP flood

2. Resource Depletion:

- Exhaust server resources: CPU, memory, connections
- Server cannot serve new requests
- Example: SYN flood, HTTP request flood

3. Application Vulnerability Exploitation:

- Exploit software bugs to crash service
- Send malformed packets, trigger error conditions
- Example: Ping of Death, teardrop attack

6.2 Flooding Attacks

ICMP Flood (Ping Flood)

Send massive number of ICMP Echo Request packets. Victim exhausts bandwidth responding with Echo Reply.

Countermeasure: Rate limiting ICMP responses, filtering ICMP at firewall.

UDP Flood

Send large volume of UDP packets to random ports. Victim checks for listening application, sends ICMP 'Destination Unreachable' for each, consuming resources.

Countermeasure: Firewall filtering, UDP traffic shaping.

TCP SYN Flood

Exploit TCP three-way handshake:

Normal handshake:

65. Client sends SYN
66. Server responds SYN-ACK, allocates resources
67. Client sends ACK, connection established

Attack:

- Attacker sends many SYN packets with spoofed source IPs
- Server sends SYN-ACK, waits for ACK
- ACK never arrives (spoofed IP unreachable)
- Server's connection queue fills up with half-open connections
- Legitimate clients cannot connect

Countermeasures:

- SYN cookies: Don't allocate resources until ACK received
- Increase connection queue size (temporary mitigation)
- Timeout half-open connections quickly
- Firewall filtering: SYN rate limiting

HTTP Flood (Application Layer DoS)

Send legitimate-looking HTTP requests to overwhelm web server or application.

- GET requests to resource-intensive pages
- POST requests with large payloads
- Requests to database-backed pages
- Harder to detect than network floods (appears as normal traffic)

Countermeasure: Rate limiting per IP, CAPTCHA challenges, web application firewall.

6.3 Distributed Denial-of-Service (DDoS)

DDoS Architecture

Attacker uses botnet to launch coordinated attack from multiple sources.

Components:

- Attacker: Controls the attack
- Master/Handler: C&C servers that control bots
- Bots/Zombies: Compromised machines that execute attack
- Victim: Target of the attack

Why DDoS is More Dangerous

- Amplification: Attack traffic from thousands of sources
- Harder to block: Cannot simply block one IP address
- Distributed origin: Difficult to trace back to attacker
- Exceeds victim's capacity: Even large organizations can be overwhelmed

DDoS Amplification Attacks

Attacker sends requests with spoofed victim IP to reflectors (servers that respond). Reflectors send large responses to victim.

Examples:

- DNS amplification: Small DNS query → large response (amplification factor ~50x)
- NTP amplification: Exploit monlist command (amplification factor ~500x)
- Memcached amplification: UDP requests (amplification factor ~50,000x)

DDoS Mitigation

- Overprovisioning: Extra bandwidth, server capacity (expensive, may not suffice)
- Traffic filtering: Block malicious traffic at ISP level
- Rate limiting: Throttle requests per source
- Content Delivery Network (CDN): Distribute load across many servers
- DDoS protection services: Cloudflare, Akamai (absorb attack traffic)
- Anycast: Distribute attack across multiple data centers

6.4 Intrusion Detection Systems (IDS)

What is IDS?

Monitors network or system for malicious activity or policy violations. Generates alerts when suspicious behavior detected.

IDS Types

1. Network IDS (NIDS):

- Monitors network traffic
- Placed at strategic points: perimeter, internal segments
- Analyzes packets for attack signatures or anomalies
- Example: Snort, Suricata

2. Host IDS (HIDS):

- Monitors single host: logs, file system, processes
- Detects local attacks, privilege escalation
- Example: OSSEC, Tripwire

Detection Methods

1. Signature-Based (Misuse Detection):

- Matches traffic against database of known attack patterns
- Low false positives: High accuracy for known attacks
- Cannot detect zero-day attacks or novel techniques
- Requires regular signature updates

2. Anomaly-Based (Behavior-Based):

- Establishes baseline of normal behavior
- Flags deviations from baseline as suspicious
- Can detect unknown attacks
- Higher false positives: Normal but unusual behavior triggers alerts
- Requires training period to learn normal patterns

3. Stateful Protocol Analysis:

- Understands protocol states and expected behavior
- Detects violations of protocol standards
- Example: Invalid TCP state transitions

IDS vs IPS

IDS (Detection)	Passive monitoring, generates alerts. Does not block traffic.
------------------------	---

IPS (Prevention)	Active blocking, can drop malicious packets or reset connections. Inline with traffic flow.
-----------------------------	---

IDS Challenges

- False positives: Legitimate activity flagged as attack (alert fatigue)
- False negatives: Real attacks not detected
- Evasion: Attackers fragment packets, encrypt traffic to bypass detection
- Performance: High-speed networks require powerful hardware
- Signature maintenance: Constant updates needed

6.5 Honeypots

What is a Honeypot?

Decoy system designed to attract and trap attackers. Appears valuable but actually monitors attacker behavior.

Honeypot Uses

- Research: Study attack techniques, malware samples
- Diversion: Divert attacker from real systems
- Early warning: Detect reconnaissance and attacks
- Collect evidence: Gather forensic data on attackers

Honeypot Types

By interaction level:

- Low-interaction: Emulate services, limited functionality. Safe, easy to deploy. Example: Honeyd
- High-interaction: Real systems with full OS. Riskier but captures more detail. Example: Real vulnerable servers

By purpose:

- Production honeypot: Deployed in organization network to detect internal/external threats
- Research honeypot: Used by security researchers to study attacker behavior

Honeynet

Network of honeypots simulating real environment. Provides comprehensive view of attack.

6.6 Firewalls

What is a Firewall?

Security device that monitors and controls incoming and outgoing network traffic based on predetermined security rules.

Firewall Characteristics

- Choke point: All traffic passes through firewall
- Enforces access policy: Allows or denies based on rules
- Logging and auditing: Records traffic for analysis

- Immunity: Firewall itself must be resistant to attacks

Firewall Limitations

- Cannot protect against attacks bypassing it (e.g., dial-up, USB drives)
- Cannot protect against internal threats
- Cannot defend against malware in allowed traffic (need antivirus, IDS)
- Cannot protect against social engineering

6.7 Firewall Access Policy

Policy Types

1. Default Deny (Whitelist):
 - Block all traffic by default
 - Explicitly allow only necessary services
 - More secure but requires careful configuration
2. Default Allow (Blacklist):
 - Allow all traffic by default
 - Block known bad traffic
 - More permissive but less secure

Rule Ordering

Firewalls process rules top-to-bottom. First match determines action. Place specific rules before general rules.

6.8 Types of Firewalls

1. Packet-Filtering Firewall

Operates at network layer. Inspects packet headers (IP addresses, ports, protocol).

Example rule: ALLOW TCP from 192.168.1.0/24 to ANY port 80

Advantages:

- Fast, low overhead
- Simple configuration
- Transparent to users

Disadvantages:

- No application awareness
- Cannot detect attacks in payload
- Vulnerable to IP spoofing
- Difficult to configure complex rules

2. Stateful Inspection Firewall

Tracks connection state (SYN, ESTABLISHED, FIN). Allows return traffic for outbound connections without explicit rule.

Maintains state table: source IP, dest IP, source port, dest port, connection state.

Advantages over packet filtering:

- Better security: Only allows legitimate response traffic
- Prevents some spoofing attacks

3. Application-Level Gateway (Proxy Firewall)

Operates at application layer. Acts as intermediary, terminates connections from clients and initiates new connections to servers.

Functionality:

- Inspects application-layer data (HTTP headers, FTP commands)
- Can enforce application-specific policies
- Content filtering: Block malicious content, URLs
- User authentication: Authenticate before allowing access

Advantages:

- Deep inspection of application data
- Strong security
- Detailed logging

Disadvantages:

- Performance overhead: Terminates and re-establishes connections
- Not transparent: May require client configuration
- Limited protocol support: Needs proxy for each application

4. Circuit-Level Gateway

Operates at session layer. Validates TCP handshake, then relays packets without inspecting payload.

Use case: SOCKS proxy for generic TCP/UDP forwarding.

5. Next-Generation Firewall (NGFW)

Combines traditional firewall with additional features:

- Deep packet inspection (DPI)
- Intrusion prevention (IPS)
- Application awareness and control
- User identity-based policies
- Malware detection
- SSL/TLS inspection

6.9 Firewall Deployment: DMZ Architecture

Demilitarized Zone (DMZ)

Isolated network segment that sits between internal network and external network (Internet).

Purpose: Host public-facing services (web server, email server, DNS) while protecting internal network.

Architecture:

- External firewall: Between Internet and DMZ
- Internal firewall: Between DMZ and internal network
- Public servers in DMZ: If compromised, attacker still separated from internal network

Rules:

- Internet → DMZ: Allow specific services (HTTP/80, HTTPS/443)

- Internet → Internal: Deny all
- DMZ → Internal: Deny most, allow only necessary (e.g., DMZ web server to internal database on specific port)
- Internal → DMZ: Allow management traffic
- Internal → Internet: Allow outbound with inspection

6.10 Public-Key Infrastructure (PKI)

What is PKI?

Framework for managing public-key encryption and digital certificates. Provides authentication, confidentiality, integrity, non-repudiation.

PKI Components

1. Certificate Authority (CA):
 - Trusted third party that issues digital certificates
 - Verifies identity before issuing certificate
 - Signs certificates with its private key
 - Examples: DigiCert, Let's Encrypt, Comodo
2. Registration Authority (RA):
 - Intermediary between user and CA
 - Verifies user identity on behalf of CA
 - Forwards certificate requests to CA
3. Digital Certificate:
 - Binds public key to identity (person, organization, website)
 - Contains: Subject name, public key, issuer (CA), validity period, signature
 - Standard format: X.509
4. Certificate Revocation List (CRL):
 - List of revoked certificates (compromised, no longer trusted)
 - Published by CA
 - Checked before trusting certificate
5. Online Certificate Status Protocol (OCSP):
 - Real-time certificate validation
 - Alternative to downloading entire CRL
 - Client queries OCSP responder for specific certificate status

Certificate Trust Chain

Certificates form hierarchy:

- Root CA: Self-signed, pre-trusted by OS/browser
- Intermediate CA: Signed by root CA or another intermediate
- End-entity certificate: Signed by intermediate CA

Validation: Verify each certificate in chain up to trusted root.

PKI Use Cases

- SSL/TLS for HTTPS: Authenticate web servers, encrypt traffic

- Email security: S/MIME for encrypted, signed email
- Code signing: Verify software publisher authenticity
- VPN authentication: Authenticate VPN endpoints
- Document signing: Adobe, Microsoft Office digital signatures

PKI Challenges

- CA compromise: If CA private key stolen, attacker can issue fake certificates
- Certificate validation: Clients must check revocation status (often skipped for performance)
- Key management: Secure storage of private keys
- Certificate lifecycle: Issuance, renewal, revocation processes

Key Takeaway: Network security requires layered defenses: DDoS mitigation protects availability, IDS detects intrusions, firewalls enforce access policies, and PKI provides trusted authentication and encryption for secure communications.

UNIT 7: Wireless, Cloud, and IoT Security

Emerging technologies introduce new security challenges. This unit examines security for wireless networks, cloud computing, and Internet of Things (IoT) devices.

7.1 Cloud Computing

Cloud Service Models

IaaS	Infrastructure as a Service: Provides virtualized computing resources (VMs, storage, networks). Example: AWS EC2, Azure VMs. User controls OS, applications. Provider manages physical infrastructure.
PaaS	Platform as a Service: Provides platform for developing and deploying applications. Example: Google App Engine, Heroku. User controls applications. Provider manages OS, runtime, infrastructure.
SaaS	Software as a Service: Provides complete application over internet. Example: Gmail, Salesforce, Office 365. User only configures application. Provider manages everything.

Cloud Deployment Models

- Public Cloud: Services offered to general public, multi-tenant. Example: AWS, Azure, Google Cloud.
- Private Cloud: Dedicated to single organization. More control, higher cost.
- Hybrid Cloud: Combination of public and private. Sensitive data in private, public for scalable workloads.
- Community Cloud: Shared by organizations with common concerns (e.g., government agencies).

7.2 Cloud Security Concepts

Shared Responsibility Model

Security responsibilities divided between cloud provider and customer. Varies by service model:

IaaS:

- Provider: Physical security, network infrastructure, hypervisor
- Customer: OS patching, application security, data encryption, access control

PaaS:

- Provider: Infrastructure, OS, runtime, middleware
- Customer: Application code security, data protection, user access

SaaS:

- Provider: Everything except data and user management
- Customer: Data classification, user access policies, compliance

Cloud Security Threats

1. Data Breaches:

- Unauthorized access to stored data
- Multitenancy vulnerabilities: Improper isolation between tenants
- Insider threats: Malicious cloud provider employees

2. Insecure APIs:

- Cloud services accessed via APIs
- Weak authentication, lack of encryption, injection flaws
- Attackers exploit API vulnerabilities to access data or services

3. Account Hijacking:

- Phishing, credential stuffing, weak passwords
- Attacker gains access to cloud account, can manipulate data, launch attacks

4. Denial of Service:

- DDoS attacks overwhelm cloud resources
- Economic DoS: Attacker exhausts victim's cloud credits by triggering resource consumption

5. Malicious Insiders:

- Cloud provider employees with privileged access
- Can access customer data if controls insufficient

6. Data Loss:

- Accidental deletion, provider outage, natural disaster
- Need backups and disaster recovery plan

7. Insufficient Due Diligence:

- Organizations migrate to cloud without understanding security implications
- Legacy security models don't translate to cloud

7.3 Cloud Security Approaches

Data Protection

1. Encryption:

- Encrypt data at rest: Use provider's encryption or bring your own key (BYOK)
- Encrypt data in transit: TLS for all communications
- Encrypt data in use: Emerging technologies like Intel SGX, confidential computing

2. Data Loss Prevention (DLP):

- Monitor and control data movement
- Prevent sensitive data from leaving cloud environment
- Classify data based on sensitivity

3. Backup and Recovery:

- Regular backups to multiple locations
- Test recovery procedures
- Geographic redundancy

Identity and Access Management (IAM)

- Centralized identity management across cloud services
- Multi-factor authentication (MFA) for all accounts
- Role-based access control (RBAC)
- Principle of least privilege
- Regular access reviews and revocation

Network Security

- Virtual Private Cloud (VPC): Isolated network segments
- Security groups: Virtual firewalls for instances
- Network segmentation: Separate production, development, DMZ
- Intrusion detection/prevention systems (IDS/IPS)

Logging and Monitoring

- Centralized logging: Collect logs from all cloud resources
- SIEM integration: Real-time analysis and alerting
- Continuous monitoring: Detect anomalies, unauthorized access
- Audit trails: Track who accessed what and when

Compliance and Governance

- Understand regulatory requirements (GDPR, HIPAA, PCI-DSS)
- Choose cloud provider with relevant certifications (SOC 2, ISO 27001)
- Data residency: Ensure data stored in compliant jurisdictions
- Regular security assessments and audits

Vendor Management

- Review provider's security policies and practices
- Service Level Agreements (SLAs): Define uptime, data protection guarantees
- Incident response plan: Coordinate with provider
- Exit strategy: Plan for data migration if switching providers

7.4 The Internet of Things (IoT)

What is IoT?

Network of physical devices embedded with sensors, software, and connectivity to collect and exchange data. Examples: smart home devices, wearables, industrial sensors, medical devices, connected vehicles.

IoT Architecture Layers

1. Device/Perception Layer:

- Physical sensors and actuators
- Collect data (temperature, motion, video)
- Control physical processes (locks, thermostats, lights)

2. Network/Communication Layer:

- Protocols: Wi-Fi, Bluetooth, Zigbee, LoRaWAN, cellular (4G/5G)
- Gateways: Aggregate data from devices, forward to cloud

3. Processing/Middleware Layer:

- Edge computing: Process data close to devices
- Cloud backend: Data storage, analytics, machine learning

4. Application Layer:

- User interfaces: Mobile apps, web dashboards
- Business logic: Automation, alerts, control

7.5 IoT Security Challenges

Device-Level Challenges

1. Resource Constraints:

- Limited CPU, memory, battery
- Cannot run complex security software
- Weak or no encryption

2. Insecure Defaults:

- Default passwords (admin/admin) not changed
- Unnecessary services enabled
- No firmware update mechanism

3. Physical Security:

- Devices deployed in accessible locations
- Tampering, theft possible
- Attackers can extract credentials, firmware

4. Long Lifespan:

- Devices used for years without updates
- Become vulnerable as new attacks discovered
- Vendors may stop supporting old devices

Network-Level Challenges

1. Insecure Communication:

- Unencrypted protocols (HTTP, MQTT without TLS)
- Eavesdropping, man-in-the-middle attacks possible

2. Weak Authentication:

- No mutual authentication between device and server
- Replay attacks, rogue device injection

3. DDoS Botnets:

- Compromised IoT devices used in massive DDoS attacks
- Example: Mirai botnet (2016) - infected 600,000 devices, took down major websites

Backend/Cloud Challenges

1. Data Privacy:

- IoT devices collect sensitive personal data
- Location tracking, health data, video/audio
- Improper handling violates privacy regulations

2. Insecure APIs:

- Mobile apps, web interfaces communicate with cloud via APIs
- Vulnerabilities allow unauthorized access to device controls or data

3. Insufficient Data Protection:

- Data stored in cloud without encryption
- Inadequate access controls

7.6 IoT Security Approaches

Secure Device Design

1. Hardware Security:

- Secure boot: Verify firmware integrity before execution
- Hardware Root of Trust: Cryptographic keys stored in tamper-resistant chip
- Physical tamper detection: Alert or self-destruct if opened

2. Secure Firmware:

- Code signing: Ensure firmware from trusted source
- Over-the-air (OTA) updates: Securely deploy patches
- Rollback protection: Prevent downgrade to vulnerable versions

3. Strong Authentication:

- Unique credentials per device (not shared default passwords)
- Certificate-based authentication
- Multi-factor authentication for user access

Secure Communication

- Encrypt all data in transit: TLS/DTLS for TCP/UDP
- Lightweight protocols: TLS 1.3, DTLS for constrained devices
- Mutual authentication: Device and server verify each other
- Secure protocols: MQTT over TLS, CoAP with DTLS

Network Security

- Network segmentation: Isolate IoT devices from corporate network
- Firewalls: Control traffic to/from IoT devices
- Intrusion detection: Monitor for anomalous device behavior
- Disable unnecessary services: Close unused ports, disable UPnP

Cloud/Backend Security

- API security: Authentication, rate limiting, input validation
- Data encryption: Encrypt IoT data at rest in cloud
- Access control: Limit who can control devices or view data
- Regular security assessments: Penetration testing, vulnerability scanning

Privacy Protection

- Data minimization: Collect only necessary data
- Anonymization: Remove PII where possible

- User consent: Transparent privacy policies, opt-in for data collection
- Compliance: GDPR, CCPA requirements for IoT data

Lifecycle Management

- Regular updates: Patch vulnerabilities promptly
- End-of-life planning: Decommission devices securely, revoke credentials
- Inventory management: Track all deployed devices, their firmware versions

7.7 Wireless Security Overview

Wireless Security Threats

1. Eavesdropping:
 - Wireless signals broadcast in all directions
 - Attacker with antenna can intercept traffic
 - Unencrypted Wi-Fi exposes all data
2. Unauthorized Access:
 - Open Wi-Fi networks allow anyone to connect
 - Weak passwords cracked (WEP, WPA with weak PSK)
 - Rogue access points: Attacker sets up fake AP with legitimate-sounding name
3. Man-in-the-Middle:
 - Evil twin attack: Fake AP impersonates legitimate one
 - Intercept traffic, steal credentials
4. Denial of Service:
 - Jamming: Radio interference disrupts Wi-Fi
 - Deauthentication attack: Flood deauth frames, disconnect clients

802.11 Security Evolution

1. WEP (Wired Equivalent Privacy):
 - Original Wi-Fi security, now deprecated
 - Uses RC4 stream cipher with 40 or 104-bit key
 - Fatally flawed: IV reuse, weak key scheduling
 - Can be cracked in minutes
2. WPA (Wi-Fi Protected Access):
 - Interim solution after WEP broken
 - TKIP (Temporal Key Integrity Protocol): Per-packet keys
 - Still vulnerable, deprecated
3. WPA2:
 - Based on 802.11i standard (2004)
 - Uses AES encryption with CCMP (Counter Mode CBC-MAC Protocol)
 - Strong security if strong passphrase used
 - Personal mode (PSK): Shared password
 - Enterprise mode (802.1X): RADIUS server authentication
4. WPA3 (2018):

- Stronger encryption: 192-bit security suite for Enterprise
- SAE (Simultaneous Authentication of Equals): Replaces PSK, resistant to offline dictionary attacks
- Forward secrecy: Past traffic cannot be decrypted even if password later compromised
- Improved IoT security: Easy Connect for devices without displays

WPA2/WPA3 Security Mechanisms

1. Four-Way Handshake:

Establishes session keys between client and AP without transmitting key over air.

68. AP sends nonce to client
69. Client generates Pairwise Transient Key (PTK), sends nonce + MIC
70. AP sends GTK (Group Temporal Key) encrypted with PTK
71. Client confirms receipt

2. CCMP (Counter Mode with CBC-MAC Protocol):

- Provides confidentiality (AES-CTR) and integrity (CBC-MAC)
- Per-packet encryption with unique IV
- Prevents replay attacks

Wireless Security Best Practices

- Use WPA3 if available, WPA2 minimum (never WEP/WPA)
- Strong passphrase: 20+ characters, random
- Change default SSID and admin credentials
- Disable WPS (Wi-Fi Protected Setup): Vulnerable to brute force
- Hide SSID: Minor security through obscurity (easily defeated but reduces casual discovery)
- MAC filtering: Allow only known devices (can be bypassed but adds layer)
- Enable firewall on wireless router
- Guest network: Isolate guest traffic from internal network
- Regular firmware updates: Patch vulnerabilities

Enterprise Wireless Security (802.1X)

- RADIUS server centrally manages authentication
- Each user has unique credentials
- Supports EAP (Extensible Authentication Protocol) methods:
 - - EAP-TLS: Certificate-based, strongest
 - - EAP-PEAP: Protected EAP, password-based but tunneled
 - - EAP-TTLS: Tunneled TLS
- Avoid LEAP: Cisco proprietary, deprecated, vulnerable

7.8 Mobile Device Security

Mobile Security Threats

1. Malicious Apps:

- Trojanized apps in official stores or third-party sources
- Steal data, send premium SMS, display ads, join botnets

2. OS and App Vulnerabilities:

- Unpatched devices at risk
- Privilege escalation, remote code execution

3. Network Attacks:

- Public Wi-Fi eavesdropping
- Fake cellular base stations (IMSI catchers, Stingrays)
- SMS phishing (smishing)

4. Data Leakage:

- Apps with excessive permissions access contacts, location, photos
- Clipboard hijacking: Malware reads copied passwords
- Cloud backup exposures

5. Physical Theft/Loss:

- Lost or stolen device contains sensitive data
- Lack of encryption or screen lock allows access

Mobile Security Best Practices

User-Level:

- Keep OS and apps updated
- Download apps only from official stores (Google Play, App Store)
- Review app permissions: Deny unnecessary access
- Use strong screen lock: PIN, password, biometric
- Enable full-disk encryption (usually default on modern devices)
- Install reputable mobile security app
- Avoid public Wi-Fi for sensitive transactions (use VPN if needed)
- Enable Find My Device / Find My iPhone for remote wipe
- Regular backups

Enterprise-Level (MDM - Mobile Device Management):

- Enforce security policies: Require encryption, screen lock, password complexity
- Remote wipe: Erase device if lost or employee leaves
- App whitelisting/blacklisting: Control which apps can be installed
- Containerization: Separate work and personal data
- VPN enforcement: Require VPN for corporate access
- Detect jailbroken/rooted devices: Block from corporate network
- Monitoring and auditing: Track device compliance

iOS vs Android Security

iOS:

- Walled garden: Apps vetted by Apple, less malware
- Regular, timely updates pushed to all devices
- Sandboxing: Apps isolated from each other and OS
- Hardware-backed encryption (Secure Enclave)

Android:

- Open ecosystem: More flexibility, more risk

- Fragmented updates: Many devices slow to receive patches (vendor-dependent)
- Google Play Protect: Scans apps for malware
- More customization allows power users to harden security
- Allows sideloading: Installing apps from outside Play Store (security risk)

Key Takeaway: Securing emerging technologies (cloud, IoT, wireless, mobile) requires understanding their unique architectures and threats. Defense strategies include strong authentication, encryption, network segmentation, regular updates, and user education. The shared responsibility model in cloud and lifecycle management in IoT are critical concepts.